

VERILOG · HOMEWORK

100 MHz 클럭 + 10 Hz Tick으로 만든 0 ~ 9999 카운터 · 4-Digit FND 표시

RTL 설계 · 합성 · 시뮬레이션 · Basys3 보드 검증

v1 Tick 카운터 → v2 스위치 제어 (CLEAR > STOP > DOWN)

v1 Tick Counter → v2 Switch Priority → Boundary Verified

v1 Tick 카운터

- tick_gen: 1천만 클럭마다 1클럭 펄스
- tick_counter: Tick마다 +1, 9999 다음 0
- FND_CONTROLLER: 기존 모듈 재사용
- 검증: Tick 주기, 끝값, 자리 분리



v2 스위치 제어

- 스위치 3개: STOP · CLEAR · DOWN
- 우선순위: CLEAR > STOP > DOWN
- 8가지 조합을 표 하나로 판정
- 검증: 보드 — CLEAR 4조합, STOP, DOWN

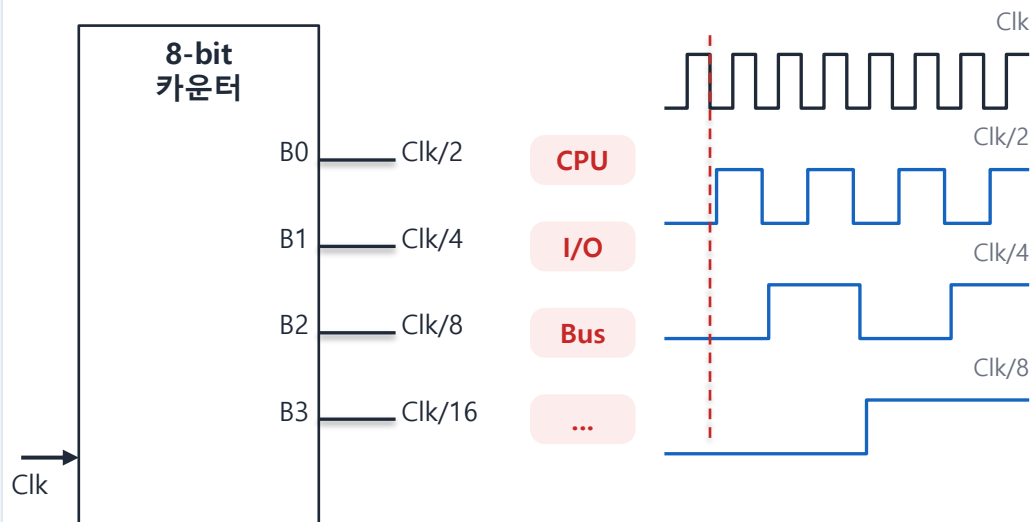


Next 검증 보강

- 남은 경우: Tick 사이 스위치 변화
- 기대값과 자동 비교하는 TB
- Fnd_Controller도 Tick 방식으로 통일
- 스위치 입력 동기화

Clock IPs Align Edges Through a Dedicated Clock Tree

카운터 비트를 클럭으로 나눠 쓰는 방식



문제

- 카운터 출력(Q)은 FF 지연 + 일반 배선을 거쳐 나감 → 비트마다 엣지가 조금씩 늦게 도착 (**Skew**)
- 도구가 이 신호를 클럭으로 관리하지 못할 경우 타이밍 검사에서 빠짐

1 클럭 타이밍 조절 방법

- 클럭 IP(MMCM/PLL)는 기준 클럭 하나로 고속 VCO를 만든 뒤 나눠서 출력 → 모든 출력이 같은 기준 엣지에서 나옴
- 피드백 경로로 클럭 버퍼 지연까지 측정해 입력 클럭과 위상을 맞춤 (Deskew)

2 Clock Skew 관리 방법

- 물리: 전용 클럭 버퍼(BUFG) + 칩 전체에 길이를 맞춘 클럭 트리 → FF마다 도착 시간이 거의 같음
- 도구: 타이밍 분석이 FF마다 클럭 도착 시간을 계산해 Slack에 반영 → 리포트의 Clock Skew 항목으로 확인

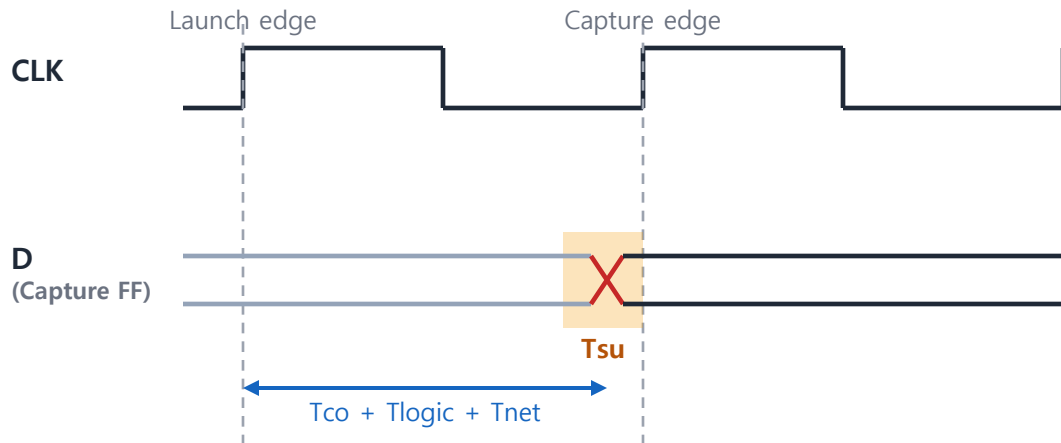
3 클럭 IP 내부 구조



$$f_{\text{out}} = f_{\text{in}} \times M \div (D \times O) \cdot \text{위상 일치 시 LOCKED} = 1 \rightarrow \text{이후 사용}$$

Setup Violation — Data Arrives Too Late for the Capture Edge

Setup Violation Timing



데이터가 T_{su} 구간 안에 도착 → Setup violation

정의

- Capture 엣지 기준 T_{su} 이전까지 데이터 도착 필요
- 도착이 늦음 → Setup Slack < 0

치명적인 이유

- Capture FF가 이전 값 · 중간 전압을 샘플 → 잘못된 값 저장
- Metastability → 다음 단까지 오류 전파
- RTL 시뮬레이션에서는 안 보이고 보드에서만 간헐적 오동작

발생 원인

- 조합 logic 단수가 많은 긴 critical path
- 긴 net · 큰 fan-out → 배선 지연
- 목표 클럭 주파수 과다 (T_{clk} 부족)
- Capture 클럭이 일찍 도착하는 clock skew

해결 방법

- Pipelining — register 삽입으로 경로 분할
- logic 최적화 · retiming
- register duplication으로 fan-out 감소
- 클럭 주파수 하향 (최후 수단)

Setup Slack

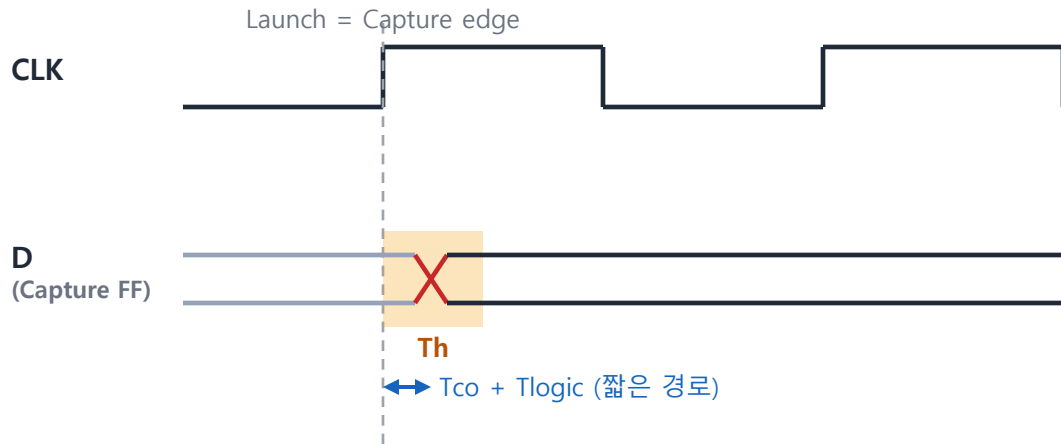
$$= (T_{clk} + T_{skew}) - (T_{co} + T_{logic} + T_{net} + T_{su})$$

Slack < 0 → Negative Slack = Setup violation

WNS = 가장 작은 Setup Slack · TNS = Negative Slack의 합

Hold Violation — New Data Arrives Too Soon After the Edge

Hold Violation Timing



새 데이터가 Th 구간 안에 도착 → 이전 값 덮어씀

정의

- Capture 엣지 이후 Th 동안 데이터 유지 필요
- 새 데이터 도착이 빠름 → Hold Slack < 0

치명적인 이유

- 같은 엣지에서 다음 값이 밀려 들어와 현재 값 덮어씀
- 클럭 주파수와 무관 → 클럭을 늦춰도 해결 불가
- 모든 동작 속도에서 실패

발생 원인

- FF → FF 직결처럼 logic이 거의 없는 짧은 경로
- Capture 클럭이 늦게 도착하는 큰 clock skew
- 일반 배선으로 만든 분주 · gated 클럭

해결 방법

- 짧은 경로에 delay(buffer) 삽입 — P&R 도구의 hold fixing
- 전용 클럭 버퍼(BUFG) · 클럭 트리로 skew 축소
- 분주 클럭 대신 Clock Enable(Tick) → 클럭 1개 유지

Hold Slack

$$= (T_{co} + T_{logic} + T_{net}) - (T_h + T_{skew})$$

Slack < 0 → Hold violation

WHS = 가장 작은 Hold Slack · THS = Negative Hold Slack의 합

All 71 Paths Pass at 100 MHz — Worst Slack +4.652 ns

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.652 ns	Worst Hold Slack (WHS): 0.247 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 71	Total Number of Endpoints: 71	Total Number of Endpoints: 58
All user specified timing constraints are met.		

+4.652 ns

Setup 최소 여유 (10 ns 주기 중)

Setup

- 데이터가 다음 클럭 엣지 전에 도착하는지 검사
- **WNS 4.652 ns** — 가장 작은 Setup Slack
- **TNS 0** — Negative Slack의 합. 0 = 위반 없음
- **Failing 0 / 71** — 검사한 도착점 71개 모두 통과

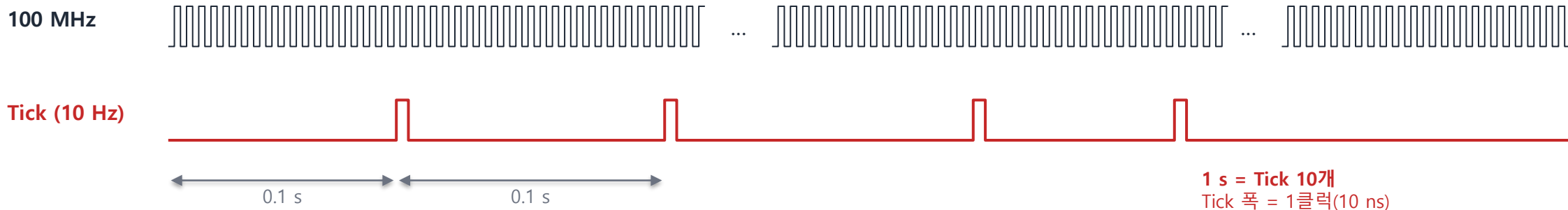
Hold

- 클럭 엣지 직후 데이터가 너무 빨리 바뀌어 직전 값을 덮지 않는지 검사
- **WHS 0.247 ns** — 가장 작은 Hold Slack
- **THS 0 · Failing 0** — 위반 없음
- 클럭을 늦춰도 안 고쳐지는 위반이라 따로 봄

Pulse Width

- 클럭 High/Low 폭이 FF · 클럭 버퍼의 최소 요구 폭 이상인지 검사
- **WPWS 4.500 ns** — 5 ns 반주기 중 4.5 ns 여유
- **TPWS 0 · Failing 0 / 58** — 위반 없음

A Tick Keeps the Whole Design on One Clock



X 분주 클럭으로 카운터를 돌릴 경우

- 분주 출력은 일반 배선 → Skew가 큼 (Q1)
- 클럭 도메인이 늘어남 → 도메인 사이 신호 전달 시 메타스테이블 위험
- 도구가 새 클럭을 모를 경우 그 경로는 타이밍 검사 누락
- 조합 논리로 만든 클럭은 Glitch → 의도치 않은 엣지
- 전용 클럭 버퍼 수가 제한됨

✓ Tick을 쓸 경우

- 모든 FF가 100 MHz 클럭 1개 → 전 경로가 타이밍 검사 대상
- Tick = 1클럭 폭 '일 허가' 신호 → FF의 CE 핀으로 연결됨 (p.12)
- 100 MHz 자원은 그대로 쓰고, 일의 단위(0.1초)만 정의
- 주기는 숫자 하나(끝값)로 변경, 여러 주기 Tick도 같은 클럭에서

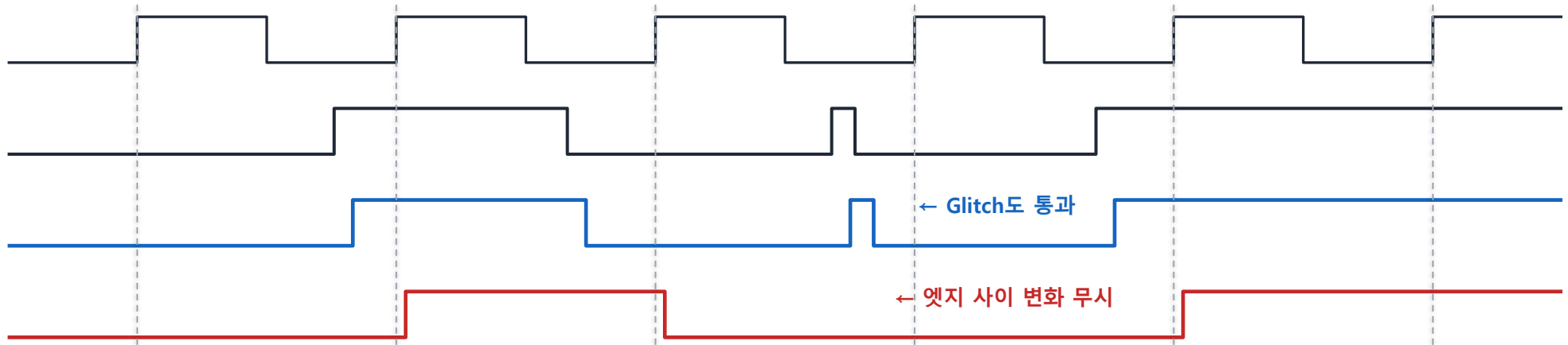
Combinational Reacts at Once, Sequential Waits for the Edge

클럭

입력

조합 출력

순차 출력



회색 점선 = 클럭 상승 엣지 — 순차 출력은 이 순간에만 바뀌고, 조합 출력은 입력 직후(전파 지연) 바로 바뀜

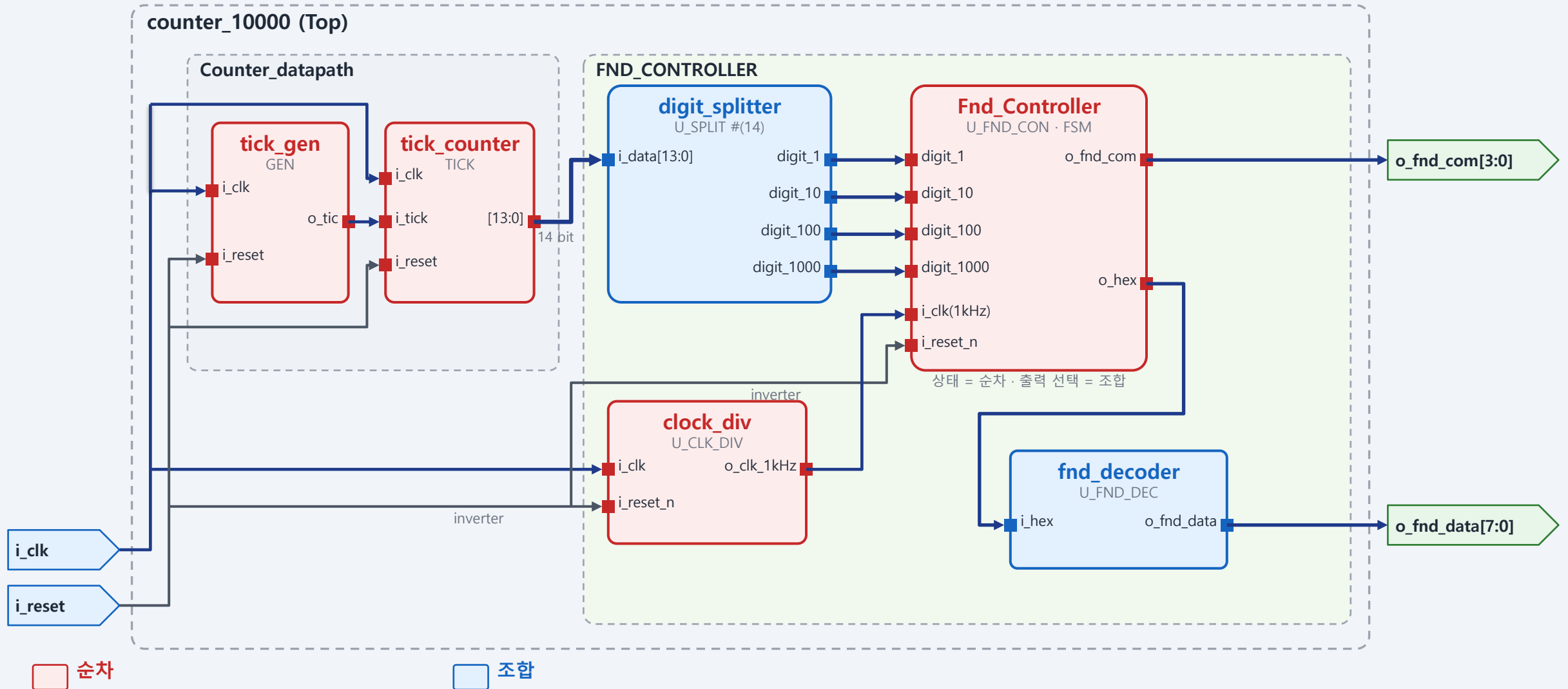
조합 회로

- 출력 = $f(\text{현재 입력})$
- 입력 변화 시 전파 지연 후 즉시 출력 변화, 중간 Glitch 가능성

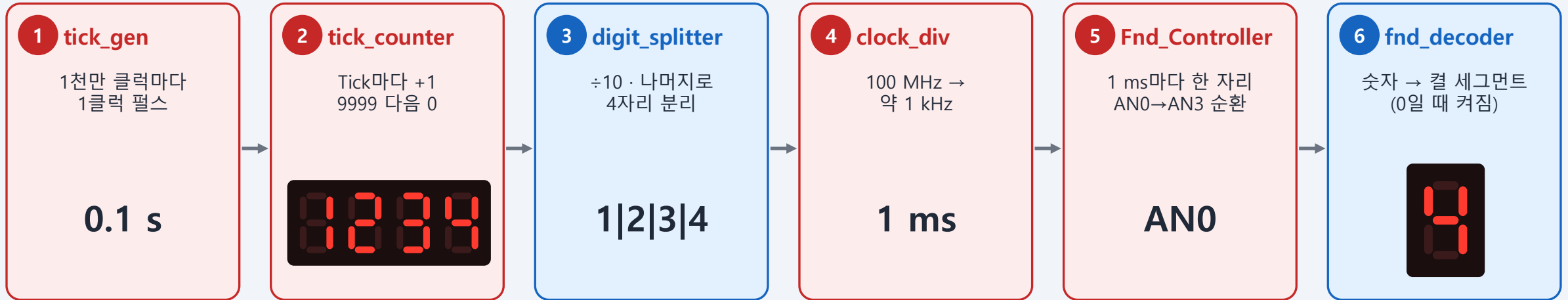
순차 회로

- 출력 = 클럭 엣지 이후 이전에 저장된 값 출력
- 클럭 엣지에서만 갱신

Top Block Diagram — Red Sequential, Blue Combinational

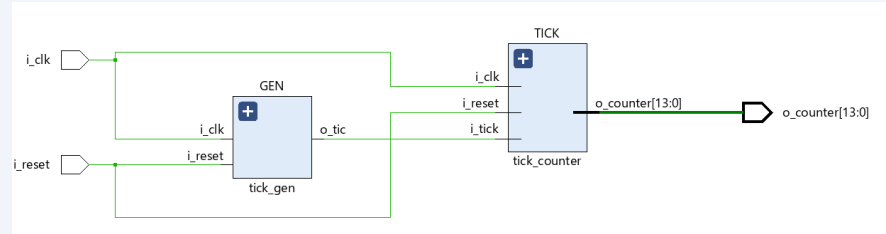


Module Roles — Count 1234 from Tick to Segments

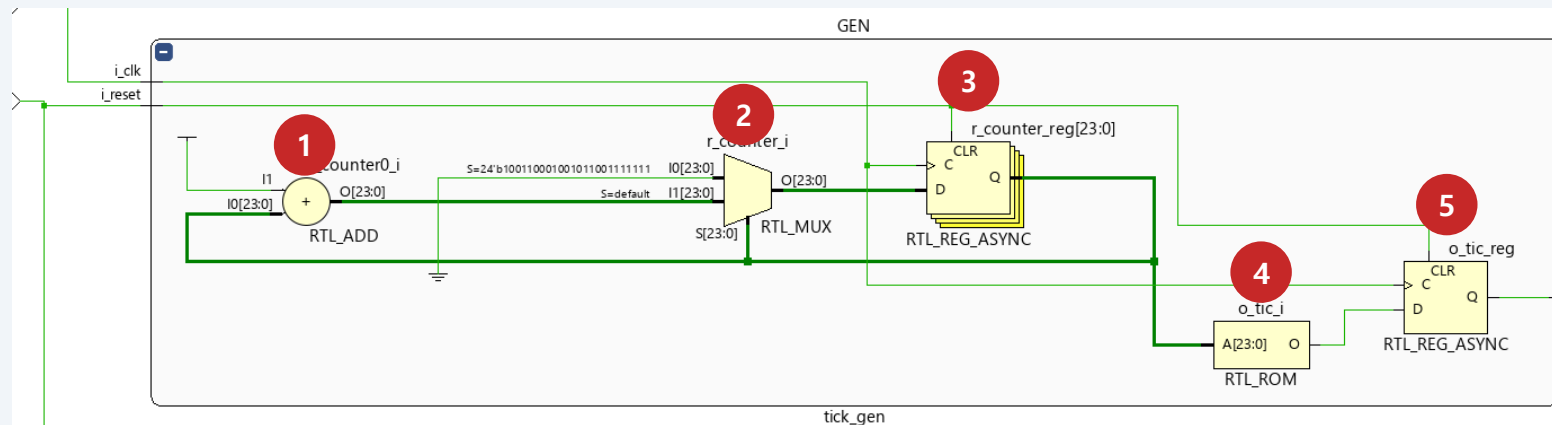


	모듈	종류	하는 일	설계 이유
①	tick_gen	순차	0 ~ 9,999,999를 세고 끝값에서 1클럭 펄스	카운터는 100 MHz 그대로 두고 '0.1초 단위'만 만들기 위해
②	tick_counter	순차	Tick이 온 클럭에서만 +1, 9999 다음 0	14 bit(최대 16,383)가 9999를 담는 최소 폭
③	digit_splitter	조합	천·백·십·일의 자리 숫자 전환	FND는 자리마다 0~9 숫자 하나를 받기 때문
④	clock_div	순차	100 MHz를 약 1 kHz 스캔 클럭으로	4자리를 빠르게 돌려 동시에 켜진 것처럼 보이게 (잔상)
⑤	Fnd_Controller	순차	4개 상태 순환 — 상태값으로 자리 선택	세그먼트 선을 4자리가 공유 $\rightarrow$ 한 번에 한 자리만 켜
⑥	fnd_decoder	조합	숫자 0~9 $\rightarrow$ 세그먼트 8개 On/Off	공통 Anode FND라 0일 때 켜짐 (Active-Low)

Tick Generator Emits One 10 ns Pulse per 10M Clocks



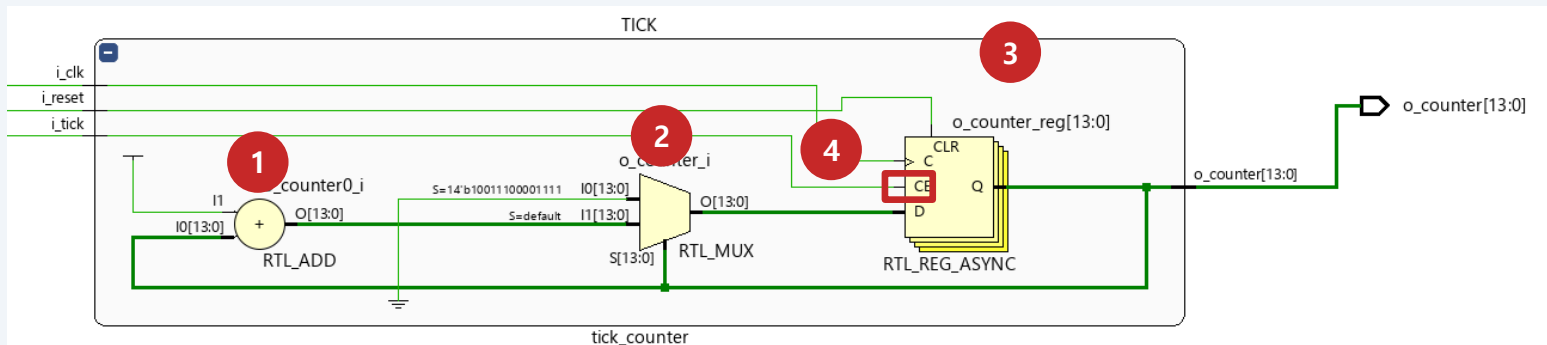
Counter_datapath = tick_gen (GEN) → Tick → tick_counter (TICK)



GEN 내부 합성 결과

- 1 **adder (+1)**
현재 값 + 1을 매 클럭 계산
- 2 **MUX**
값 = 9,999,999일 때 0, 그 외 +1 결과 → 끝에서 0으로 되돌림
- 3 **24-bit register**
 $2^{24} = 16,777,216 \geq 1\text{천만} \rightarrow 24\text{ bit}$. 리셋 버튼에 즉시 0 (비동기 CLR)
- 4 **comparator (ROM 기호)**
'값 = 끝값' 비교. 24 bit 입력 → 1 bit 출력 표라 도구가 ROM 기호로 그림
- 5 **Tick register**
비교 결과를 한 클럭 뒤 출력 → Glitch 없는 깨끗한 1 클럭(10 ns) 펄스

Tick Enters the Clock-Enable Pin, Not a Second Clock



1 **adder (+1)**
현재 값 + 1

2 **MUX**
값 = 9999일 때 0 → 0~9999 순환

3 **14-bit register**
 $2^{14} = 16,384 \geq 10,000$. 리셋 시 즉시 0

4 **CE 핀 = Tick**
Tick이 1인 클럭에서만 새 값을 받아들임. 클럭은 100 MHz 그대로, '언제 일할지'만 Tick이 결정

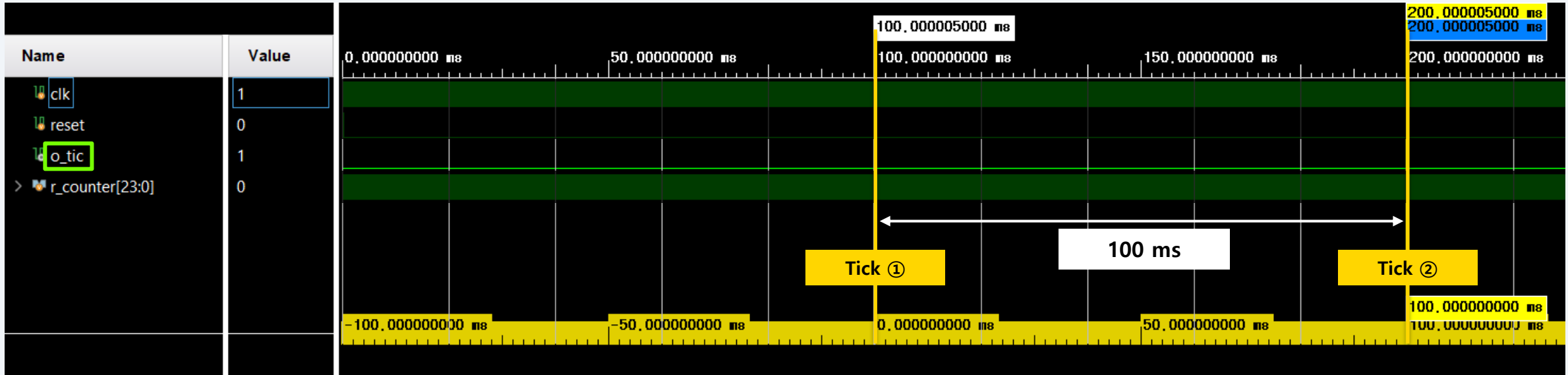
분주 클럭 방식일 경우

- register 클럭(C) 핀에 분주 신호 입력
- → 새 클럭 도메인, 일반 배선 Skew, 타이밍 검사 누락 위험

Tick 방식

- 클럭(C) 핀은 100 MHz 그대로, Tick은 CE 핀으로
- 코드는 'Tick일 때 +1'만 작성 → 합성기가 CE로 연결 — Q3 답의 증거

Tick Period Measured at 100 ms (10 Hz)



계산

- $100,000,000 \text{ Hz} \div 10 \text{ Hz} = 10,000,000$ 클럭마다 Tick 1번
- → 끝값 9,999,999

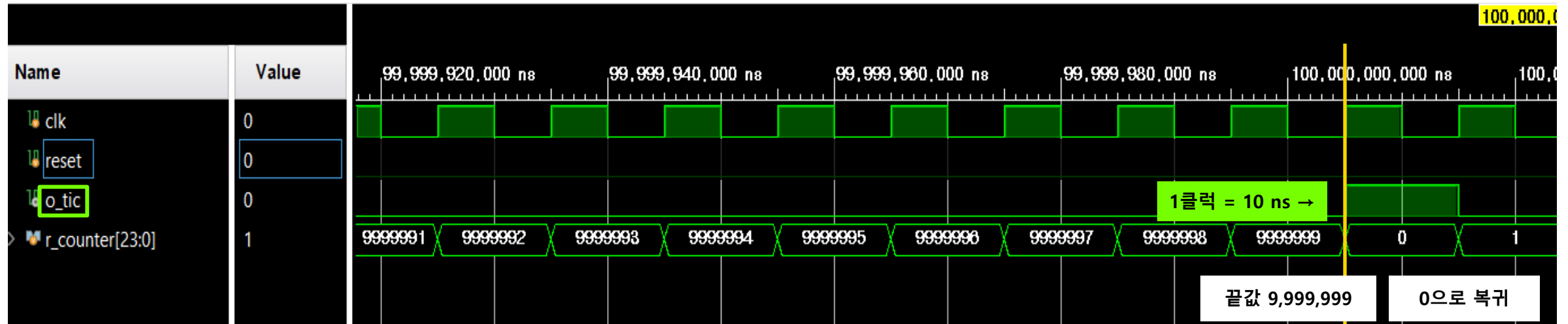
측정

- 커서 100.000005 ms → 200.000005 ms
- 간격 100.000000 ms = 10 Hz

+5 ns 이유

- 클럭 상승 엣지가 5, 15, 25 ns...에 있어 Tick도 엣지 시각에 맞춰 뜸
- 주기에는 영향 없음

9,999,999 Rolls to 0 with a One-Clock Tick



tick_gen 끝값 비교

- r_counter = 9,999,999 → comparator 출력 1
- MUX가 0 선택 → 다음 엣지에 0 복귀

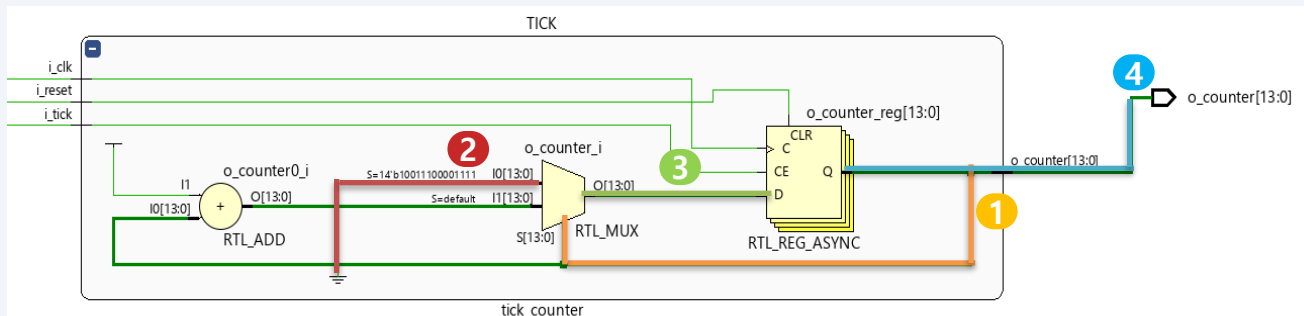
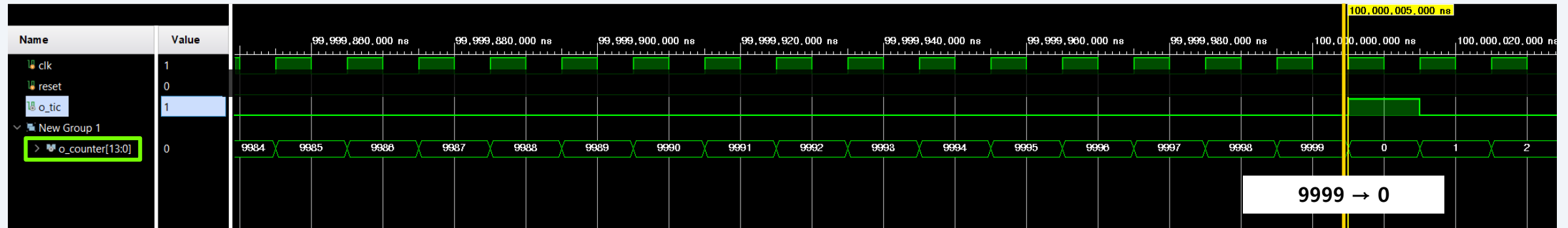
Tick register

- 비교 결과를 한 클럭 뒤 출력 → Glitch 없음
- 카운트가 0이 되는 엣지 = Tick이 1이 되는 엣지

1클럭 폭 이유

- tick_counter는 Tick = 1인 클럭마다 +1
- 1클럭(10 ns) 폭 → Tick 1번에 +1 보장

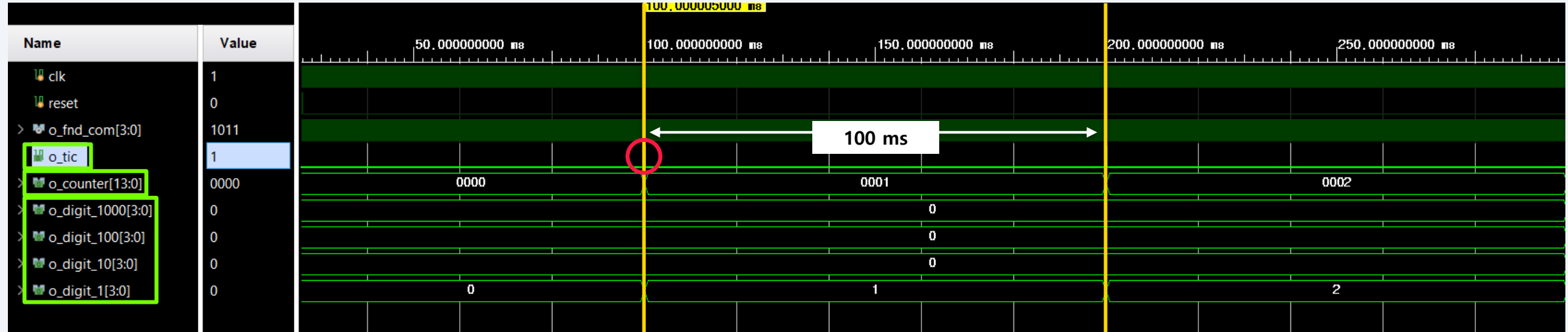
Counter Wraps from 9999 to 0



tick_counter — 9999 다음 0이 나오는 이유

- 1 comparator**
o_counter = 9999 (COUNT - 1) 감지
- 2 MUX**
+1 결과(10000) 대신 0 선택
- 3 14-bit register**
Tick 엣지에서 0 저장 → 9999 다음 0
- 4 결과**
10000은 저장되지 않음 → 0 ~ 9999 4자리 유지

Each Tick Adds 1 on the Full Top



Tick ↑ → 카운트 +1

- Tick이 뜬 클럭에서 0000 → 0001 → 0002
- 100 ms 간격 = 0.1초마다 +1

자리 분리 확인

- 1의 자리가 카운트와 같은 시각에 바뀜
- digit_splitter(조합 회로) → 클럭 대기 없음

v2 Switch Priority: CLEAR > STOP > DOWN > UP

SW0 V17

STOP
HOLD (현재 값)

SW1 V16

CLEAR
0으로

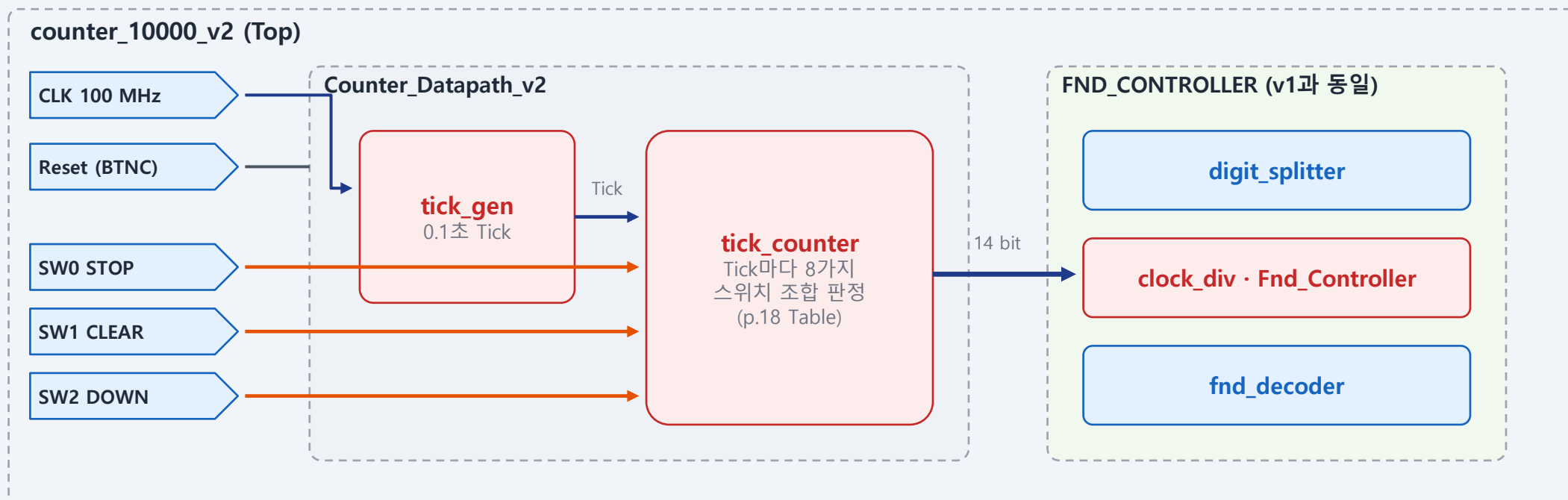
SW2 W16

DOWN
-1 (0 다음 9999)

우선순위



여러 스위치 동시 ON 시 왼쪽 우선

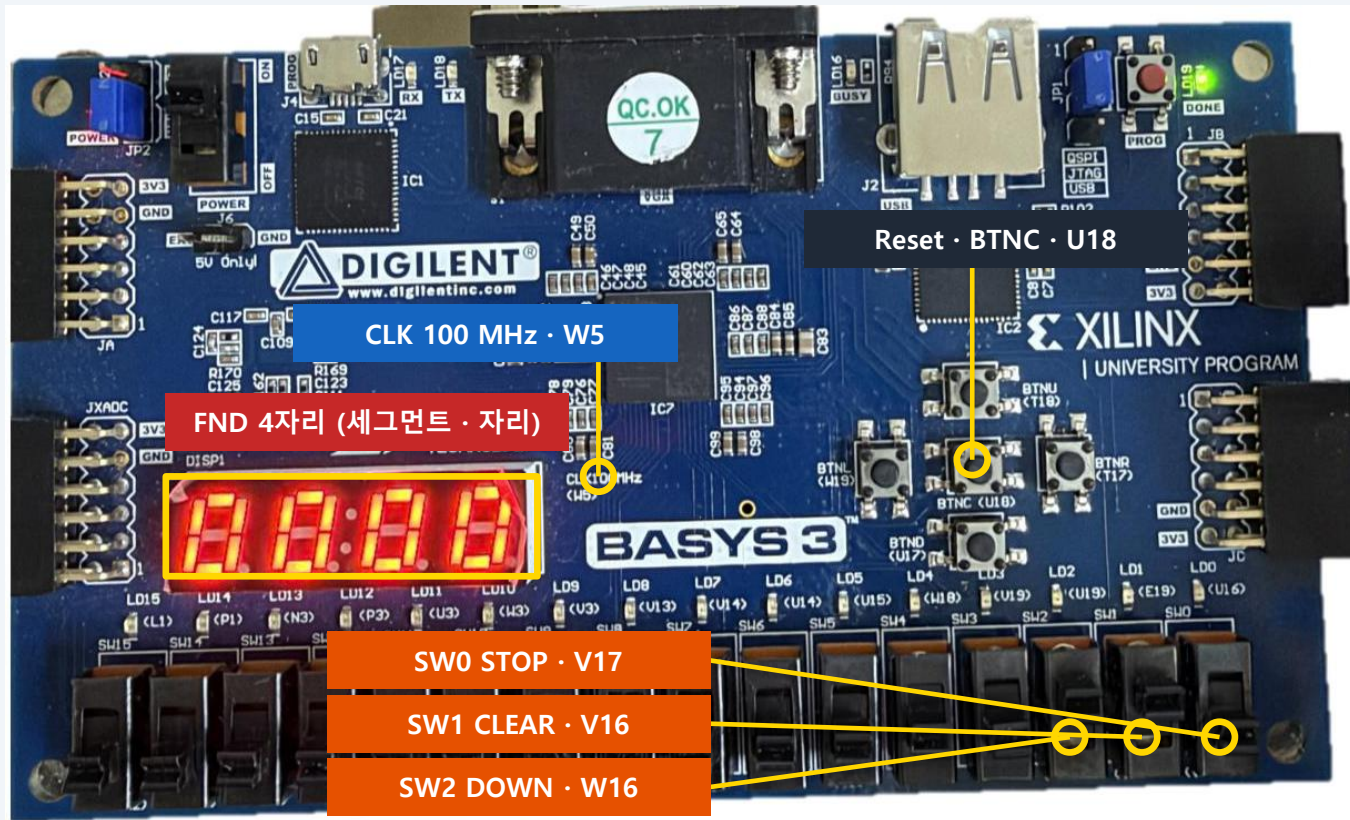


8 Switch Codes Map to 4 Actions in One Table

Switch Action Table

SW2 DOWN	SW1 CLEAR	SW0 STOP	Action	Tick 때 카운트	근거
0	0	0	UP	+1 (9999 다음 0)	스위치 없음 → 기본 동작
0	0	1	STOP	HOLD	STOP만
0	1	0	CLEAR	0	CLEAR만
0	1	1	CLEAR	0	CLEAR > STOP
1	0	0	DOWN	-1 (0 다음 9999)	DOWN만
1	0	1	STOP	HOLD	STOP > DOWN
1	1	0	CLEAR	0	CLEAR > DOWN
1	1	1	CLEAR	0	CLEAR > STOP > DOWN

Board I/O Pin Map (FND_COU.xdc)

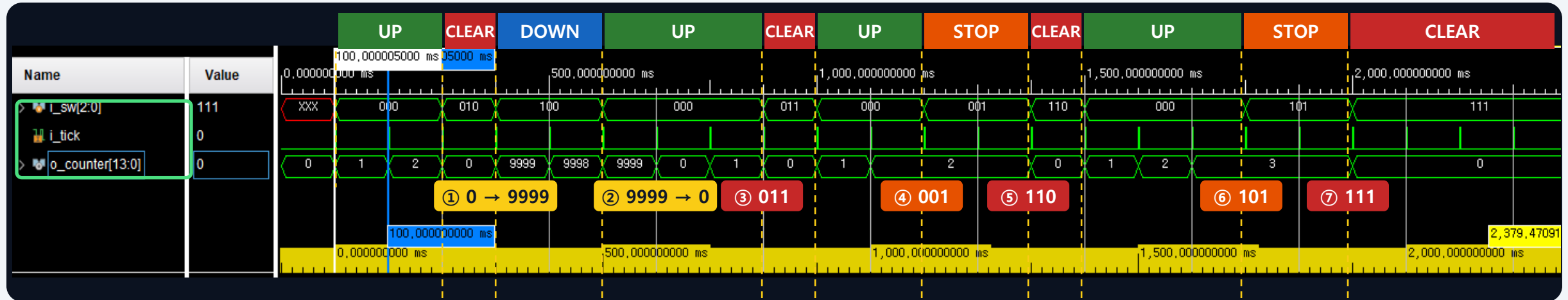


신호	보드	핀	방향
i_clk	100 MHz 오실레이터	W5	입력
i_reset	BTNC (가운데 버튼)	U18	입력
i_sw0	SW0 — STOP	V17	입력
i_sw1	SW1 — CLEAR	V16	입력
i_sw2	SW2 — DOWN	W16	입력
o_fnd_data[0:7]	세그먼트 CA~CG, DP	W7 W6 U8 V8 U5 V5 U7 V7	출력
o_fnd_com[0:3]	자리 AN0~AN3	U2 U4 V4 W4	출력

Boundary Plan — 8 of 10 Cases Confirmed in Simulation

#	확인 항목	입력 (SW2 SW1 SW0)	기대 결과	결과
1	UP 끝값	000	9998 → 9999 → 0	✓ p.22
2	DOWN 끝값	100	1 → 0 → 9999	✓ p.22
3	CLEAR	010	다음 Tick에 0 (최대 0.1초 지연)	✓ p.22
4	CLEAR > STOP	011	0	✓ p.22
5	CLEAR > DOWN	110	0	✓ p.22
6	전부 컴	111	0	✓ p.22
7	STOP	001	Tick이 와도 HOLD	✓ p.22
8	STOP > DOWN	101	HOLD	✓ p.22
9	Tick 사이 스위치 변화	Tick 사이에 켜다 끄	변화 없음 (Tick 때만 판정)	미실시
10	리셋	BTNC	즉시 0 (클럭 대기 없음)	시작 시 1회

Simulation — All 8 Switch Codes and Both Wraps Match



①② UP · DOWN 경계

- 0에서 DOWN → 9999, 이어서 9998
- UP으로 9998 → 9999 → 0 순환
- 9999까지 1,000초 대기 대신 0에서 DOWN으로 경계 직행

③⑤⑦ CLEAR 우선

- 010 · 011 · 110 · 111 모두 다음 Tick에 0
- STOP · DOWN과 함께 켜도 CLEAR 우선

④⑥ STOP 우선

- 001: 2 → 2 → 2, 2 Tick 동안 HOLD
- 101: DOWN이 켜져도 3 HOLD → STOP > DOWN

8 / 8 스위치 코드

- 000 ~ 111 전 조합 기대 동작 일치
- 경계 2곳 (0 → 9999, 9999 → 0) 확인

Scenario Breakdown — Each Step Targets One Rule

#	Tick (ms)	SW2 SW1 SW0	Action	카운트	확인 의도	결과
1	100 - 200	000	UP	0 → 1 → 2	기본 동작 — Tick마다 +1	✓
2	300	010	CLEAR	2 → 0	CLEAR 단독 — 다음 Tick에 0	✓
3	400 - 500	100	DOWN	0 → 9999 → 9998	DOWN 경계 — 0 다음 9999	✓
4	600 - 800	000	UP	9998 → 9999 → 0 → 1	UP 경계 — 9999 다음 0	✓
5	900	011	CLEAR	1 → 0	CLEAR > STOP	✓
6	1000 - 1100	000	UP	0 → 1 → 2	CLEAR 후 다시 UP	✓
7	1200 - 1300	001	STOP	2 → 2 → 2	STOP 단독 — Tick이 와도 유지	✓
8	1400	110	CLEAR	2 → 0	CLEAR > DOWN	✓
9	1500 - 1700	000	UP	0 → 1 → 2 → 3	CLEAR 후 다시 UP	✓
10	1800 - 1900	101	STOP	3 → 3 → 3	STOP > DOWN	✓
11	2000	111	CLEAR	3 → 0	CLEAR > STOP > DOWN — 전부 컴	✓

스위치 코드 8 / 8

- 000 ~ 111 전 조합 확인
- 111 = 전부 컴 → CLEAR (#11)

경계 2 / 2

- DOWN 0 → 9999 (#3)
- UP 9999 → 0 (#4)

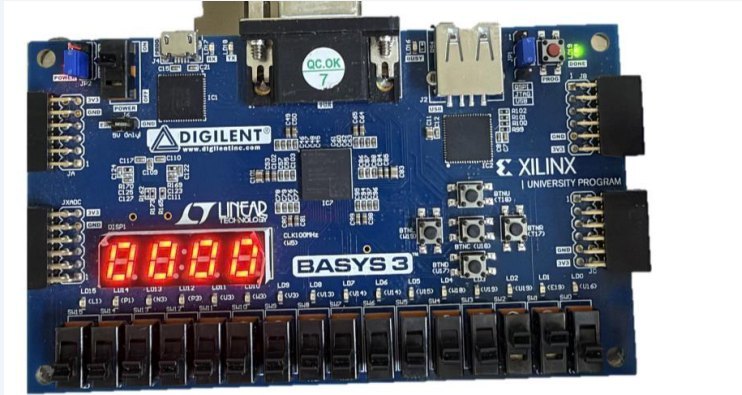
우선순위 4건

- CLEAR > STOP (#5) · CLEAR > DOWN (#8)
- STOP > DOWN (#10) · 전부 컴 → CLEAR (#11)

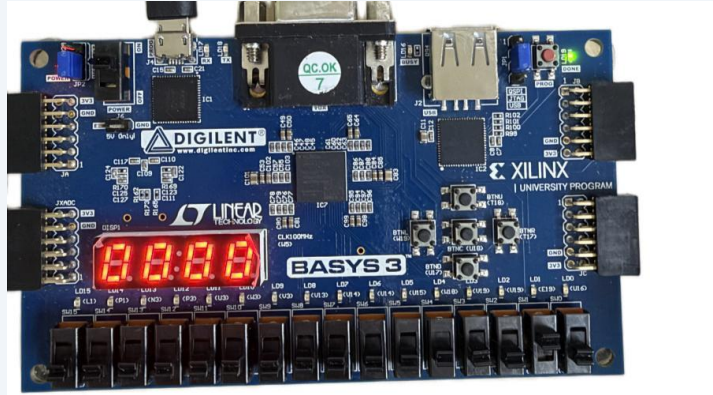
판정 방식

- 파형 · 콘솔 로그 눈 비교 (자동 비교 없음)
- → 자동 판정 TB 제안 (p.27)

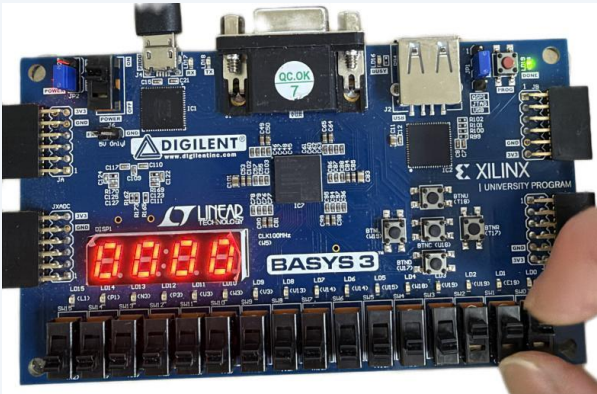
Board — CLEAR Wins in All 4 Combinations



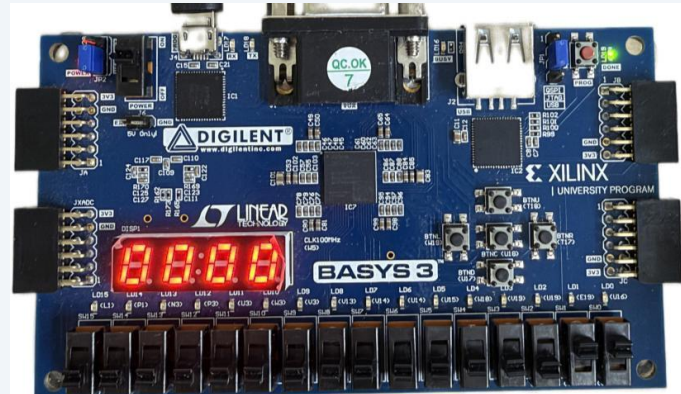
SW2 SW1 SW0 = 110 CLEAR + DOWN



SW2 SW1 SW0 = 010 CLEAR만



SW2 SW1 SW0 = 111 전부 컴



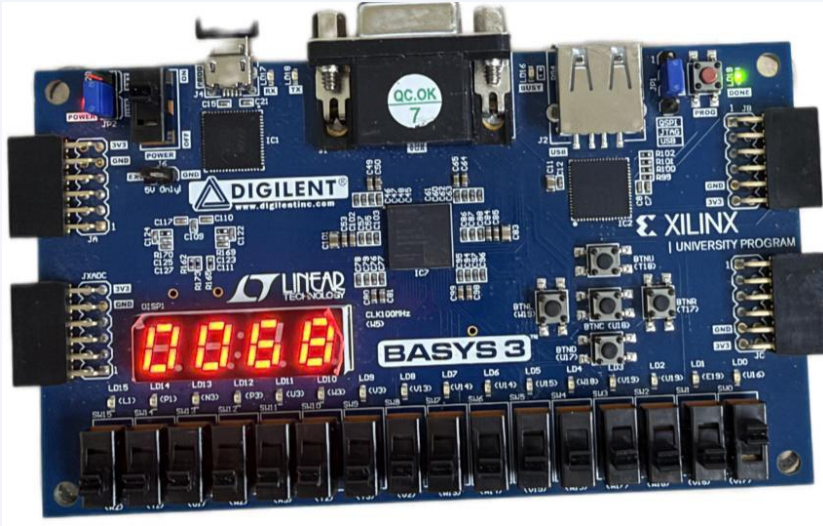
SW2 SW1 SW0 = 011 CLEAR + STOP

판정

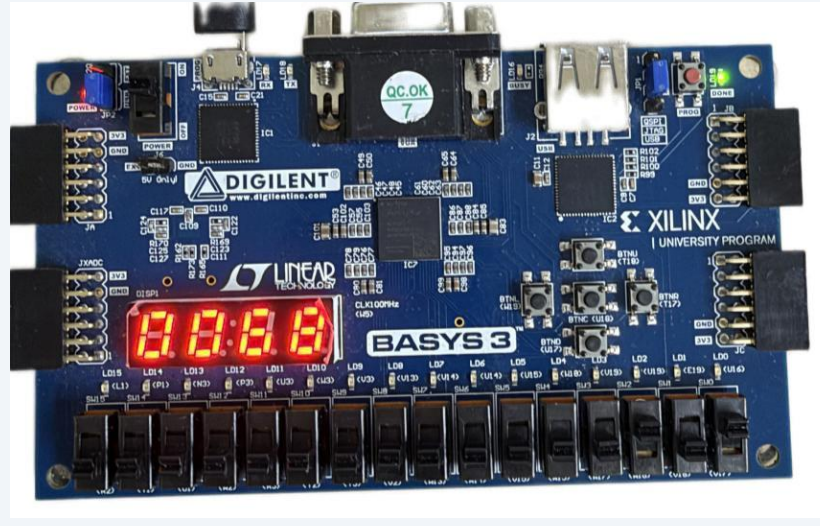
✓ 4가지 조합 모두 0000

CLEAR 스위치(SW1) ON → 다른 스위치와 상관없이 0

Board — STOP Holds the Count Even with DOWN On



SW2 SW1 SW0 = 001



SW2 SW1 SW0 = 101

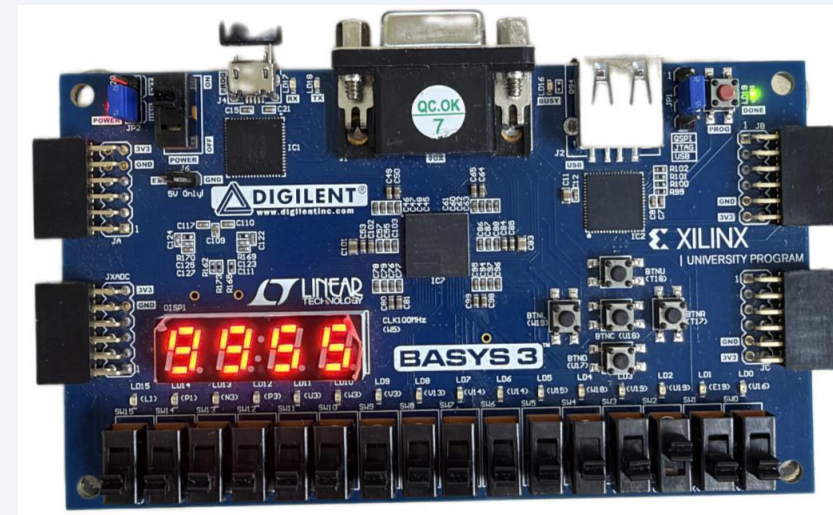
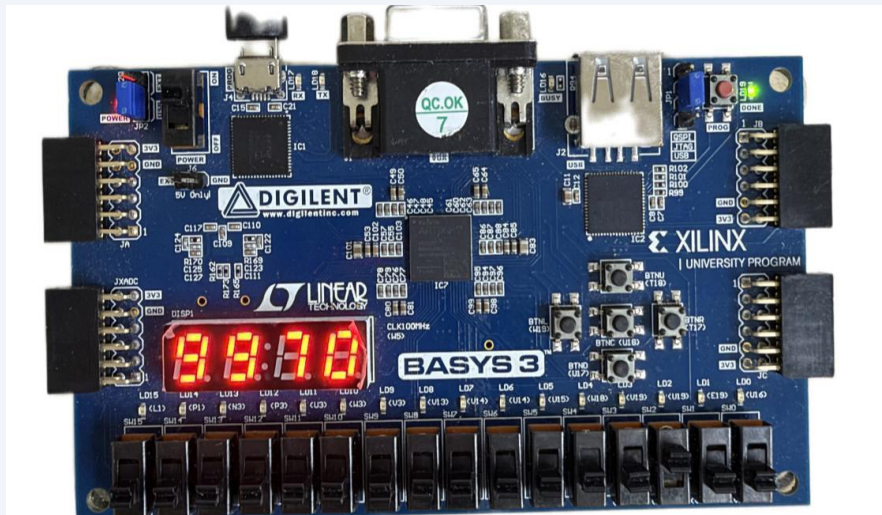
판정

✓ 101: STOP > DOWN

STOP과 DOWN이 함께 켜져도
HOLD

STOP : register CE를 끄는 방식 → Tick이 와도 HOLD

Board — DOWN Counts Back One Step per Tick



SW2 SW1 SW0 = 100 (9970 → 9955)

Verification Score 23 / 40 — Automatic Checking Missing

항목	점수	근거
입력 커버리지	●●●●○	v2 시나리오 시뮬레이션 8/8 코드 + UP · DOWN 경계 / 보드 6/8 조합
기대값 독립성	●○○○○	기대값 없이 파형을 눈으로 비교
관측 지점	●●○○○	내부 카운트 · 자리 신호 관찰, 실제 FND 출력 핀 해석 없음
판정 타이밍	●●●●○	커서로 Tick 시각(100.000005 ms)과 1클럭 폭까지 정밀 확인
자동 판정 · 리포트	●○○○○	오류 수 · PASS/FAIL 출력 없음
보드용 Top 시뮬레이션	●●●●○	v1 · v2 Top 전체 시뮬레이션 ✓
문제 해결 절차	●●●●○	우선순위를 보드 + 시뮬레이션 양쪽에서 확인
문서화 · 재현성	●●●○○	커서 시각 기록 양호 / 실제 Tick 주기로 2초 진행 → 그림

23 / 40
C+ · 자동 판정 보강 필요

Tick과 스위치 변화가 겹치는 순간은 보드로 재현 불가

Next — Self-Checking TB and Single-Clock FND Scan

1 Tick 주기를 바꿀 수 있는 시뮬레이션

- TB에서 Tick 끝값만 10으로 덮어쓰기
- 9999 경계까지 1천억 → 약 10만 클럭 (계산값)
- p.20의 10가지 경우를 몇 ms 안에 확인

2 자동 판정 TB

- 입력 → 설계와 독립된 기대값 모델 → 비교 → 오류 수 · PASS/FAIL
- FND 출력 핀(세그먼트 · 자리)을 숫자로 되돌려 비교
- 사람 눈 대신 로그가 판정

3 Fnd_Controller도 Tick으로

- 1 kHz 분주 클럭 → 100 MHz 클럭 + 1 kHz Tick(CE)
- 설계 전체가 클럭 1개 → 모든 경로가 타이밍 검사 대상
- Q3 원칙(p.7)을 설계 전체에 일관 적용

4 스위치 입력 동기화

- 스위치는 클럭과 무관하게 바뀔 → FF 2단으로 받아 메타스테이블 방지

One Clock, Many Rates — Tick = Enable

설계 결과

- 100 MHz 클럭 1개 + 0.1초 Tick으로 0~9999 카운터 완성
- 스위치 8조합을 표 하나로 판정 — CLEAR > STOP > DOWN
- 합성 결과에서 Tick이 register CE 핀으로 연결됨을 확인

검증 현황

- Tick 주기 100 ms, 1클럭 폭, 9999 → 0 시뮬레이션 확인
- v2 우선순위 시뮬레이션 8/8 코드 + 보드 6/8 조합 확인
- UP · DOWN 경계(9999 ↔ 0) 시뮬레이션 확인 — 평가 23 / 40

고찰

- Clock resource를 온전히 사용하기 위한 방법으로 tick 방식으로 모듈 분업화 스킬을 알게 됨
- 경계값에서의 동작만 확인하여 Whole Case에 대한 검증 시나리오가 필요(추후 연구)
- FND_CONTROLLER의 Anode 제어도 tick으로 구현하면 좋을 것 같음